



Travlr Getaways Website
CS 465 Project Software Design Document
Version 3.0

Table of Contents

CS 465 Project Software Design Document	1
Table of Contents	2
Document Revision History	2
Instructions	2
Executive Summary	3
Design Constraints	3
System Architecture View	4
Component Diagram	4
Sequence Diagram	6
Class Diagram	7
API Endpoints	8
The User Interface	8

Document Revision History

Version	Date	Author	Comments
1.0	07/19/2026	Andrew Allen	Initial version of summary, constraints, and system architecture.
2.0	08/02/2026	Andrew Allen	Implement sequence and class diagrams with descriptions. Also created initial API endpoint information.
3.0	08/15/2026	Andrew Allen	Added UI screenshots, endpoints, and summary.

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Travlr Getaways is requesting a travel booking website for customers to book travel packages. The website must allow users to create an account, search for travel packages by location and price point, and book reservations with their agency. Also, customers must have a way to access their trip information to see their itineraries. Travlr Getaways also requires an admin-only site where their admins can maintain their customer base, available trip packages, and pricing for all items and packages.

To address these requirements, this web application will be built using a MEAN full stack architecture, consisting of MongoDB, Express.js, Angular, and Node.js. This full stack architecture is appropriate for this web application because it provides everything needed for the client, server, and database layers while still supporting scalability and maintainability.

The application will provide a customer-facing website and an administrator single page application (SPA). The customer-facing website will display travel packages, destinations, and trip information. The pages will be dynamically rendered using Handlebars (HBS) templates and the data retrieved from the server will be in JavaScript Object Notation (JSON). The SPA will provide authenticated administrators with the ability to create, update, and manage trip information through an Angular interface.

The backend will use Node.js and Express.js to expose routes and business logic, while MongoDB will store trip information, reservations, and user account details. The websites will communicate with the backend through HTTP requests, enabling data management and delivery. This architecture directly supports the customers' requirements for a modern web application to serve customers and provide data management for its administrators.

Design Constraints

There are many design constraints to consider when developing the modern web application for Travlr Getaways.

Technology constraints:

The Travlr Getaways application is required to use the MEAN stack, which consists of MongoDB, Express.js, Angular, and Node.js. This limits the selection of programming languages, frameworks, and database technologies to be used during development. Therefore, the application must use JavaScript based technologies throughout the client, server, and database layers to maintain compatibility.

MongoDB also introduces its own constraints because it is a NoSQL document database rather than a relational database. Application data must be designed as flexible JSON documents instead of being organized into tables. This affects how travel packages, destinations, and users' information are stored.

The customer facing application uses Express.js and Handlebars (HBS) to dynamically render webpages and Angular for the single page application. Since these use two different front end approaches, I must maintain a clear separation between customer and admin views while allowing both to interact with the backend and database.

Time constraints:

As with any application, development time and deadlines are always a constraint. Limited amounts of time can affect the number of features implemented and amount of testing performed. As a result, some features may not be incorporated by the deadline, or an extension may need to be made.

Resource constraints:

This project has limited development and testing resources. As the source for all designing, development, and testing. This limits the amount of time I have for exhaustive testing. I must ensure that the app remains as modular as possible to ensure features and services can be incorporated later as needed.

Security constraints:

The application must protect user information and admin functionality from unauthorized access. Admin features must require authentication, and authorization controls should restrict access to users without access. Passwords should not be stored as plain text and should be protected using hashing methods.

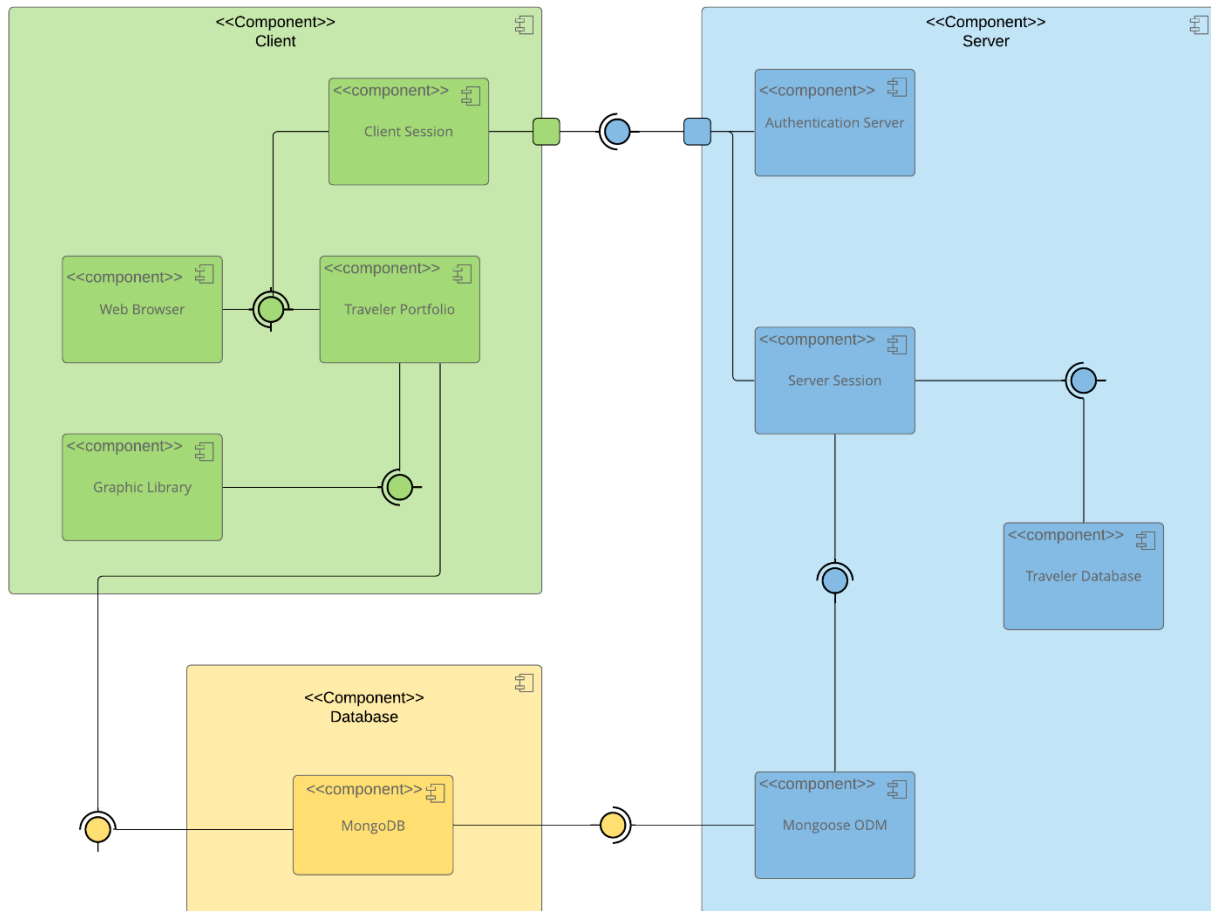
Communication between the client and server should utilize HTTPS to encrypt information transmitted over the internet. Input validation and secure database queries should be used to reduce the risk of vulnerabilities and attacks.

Since this app will collect user information, we must also be mindful of various laws and regulations like GDPR if it is serving European customers. This creates additional constraints related to user consent, data access, and data deletion.

Scalability constraints:

The application must be designed and built to support an increasing number of users, travel packages, and database records. As the data grows, inefficient database queries or poorly structured application components could reduce performance. MongoDB and Node.js support scalable web development, but I need to ensure the queries are efficient.

System Architecture View**Component Diagram**



A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version](#).

The Travlr Getaways web application uses a three-tiered architecture consisting of the client, server, and database components. Each layer has its own responsibilities and communicates with the others to provide dynamic travel information and administration functions.

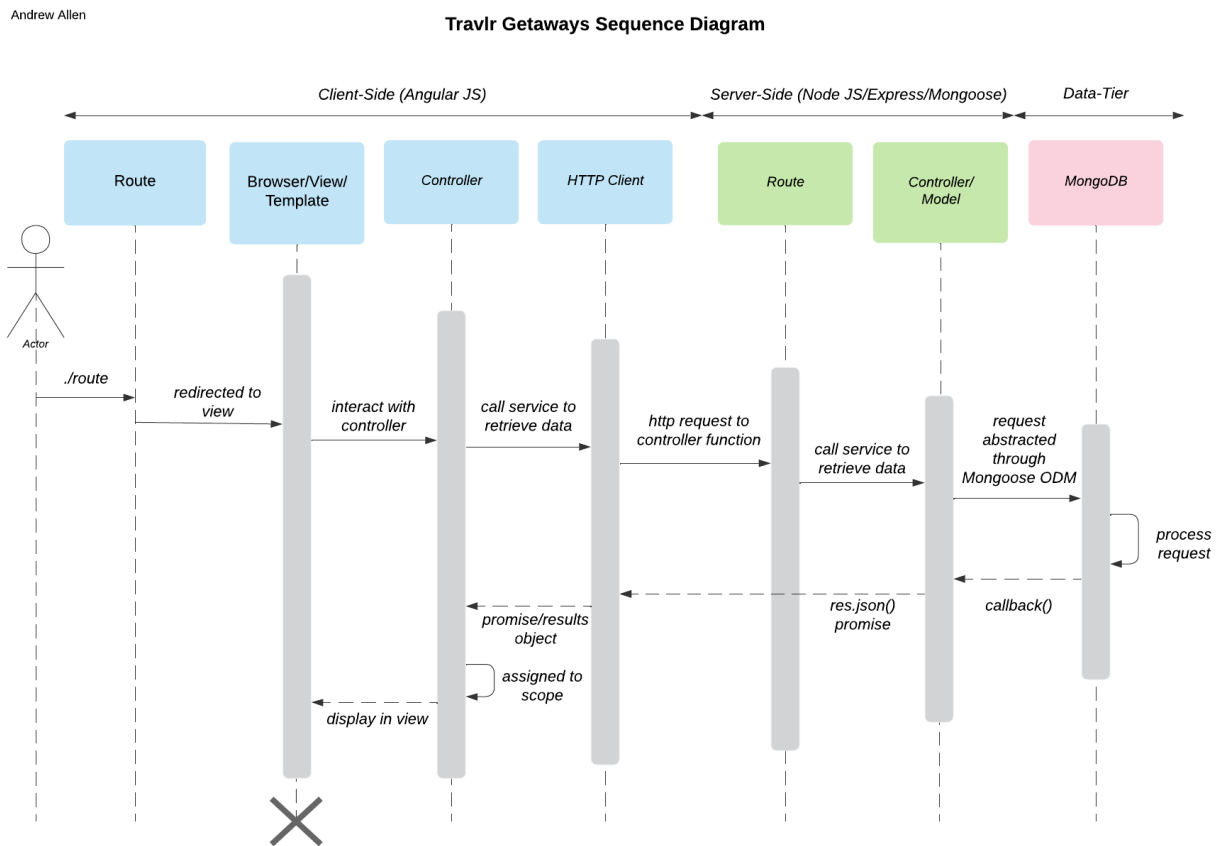
The client component contains the user interface displayed through an internet browser. Users are able gain initial access to the application through their web browsers, and the traveler portfolio component displays travel destinations and trip information. The graphic library provides all the visuals for the user interface, and the client session component communicates with the server.

The server component contains all the business' logic. The authentication server component verifies user account credentials and manages secure access for administrators. The server session component processes the incoming requests from the client, executes the apps logic, and communicates with the database. The traveler database component represents the applications data layer on the server, allowing travel information to be retrieved and updated when needed. The Mongoose ODM component is the object data modeling layer, translating JavaScript objects into MongoDB documents simplifying interactions with the database.

The database component is MongoDB, used to store trip information and user data. MongoDB communicates with the server through Mongoose enabling quick and efficient CRUD operations.

Overall, all these components together form the MEAN stack architecture. This development stack provides a separation of responsibility, improves maintainability, scalability, and performance.

Sequence Diagram



The sequence diagram above illustrates the data flow and interactions across the TravlR Getaways web application.

Interactions Overview:

A user starts by typing an address into their search bar, which is intercepted by the route. The route then redirects the request to render the appropriate view. As the user interacts with the UI, the browser/view triggers logic inside the controller. The controller invokes the HTTP client to make a request to the backend REST API. The server-side express route receives the incoming HTTP request and passes execution to the controller/model. The server controller uses Mongoose ODM to form a query and perform interactions with the MongoDB database. MongoDB processes the request and executes a callback function and returns data records. The backend controller converts the data into JSON and

resolves the HTTP promise back to the HTTP client. The client controller processes the promise and dynamically updates the browser/view/template for the user.

Trips list retrieval:

When a user opens the trips view, the client route loads the view. The client controller requests the full trip catalog using HTTP client. The requests reach the express router through `router.route('/trips').get(tripsController.tripsList)`. The controller then executes a Mongoose query to retrieve all trips records and sends them back to the view as JSON.

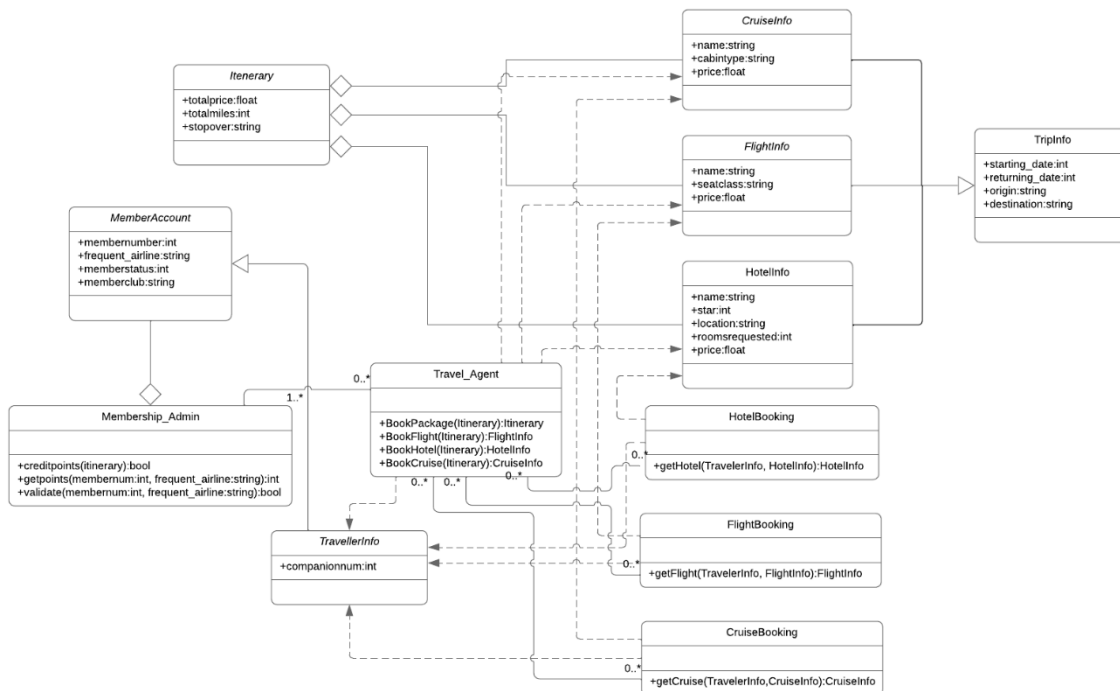
Single trip retrieval:

When a user selects a specific trip, the client controller issues a GET request with the target trip code attached to the path for example `/api/trips/GALR210214`. The backend router captures this and forwards it to `tripsController.tripsFindByCode`. The controller then queries MongoDB using `:tripcode` and returns a single trip as JSON and updates the client's view.

Class Diagram

Andrew Allen

TravlR Getaways Class Diagram



The class diagram above illustrates the structure, classes, and attributes for the TravlR Getaways web application.

FlightInfo, HotelInfo, and CruiseInfo all serve as the base classes containing general travel parameters such as starting date, returning date, origin, and destination that they inherit from the superclass TripInfo.

FlightInfo, HotelInfo, and CruiseInfo also contain their own class specific attributes such as cabin type, seat class, and rooms requested.

The Itinerary class aggregates flight, hotel, and cruise information to consolidate trips' total price, total miles, and stop over.

User information is modeled through MemberAccount and managed by Membership_Admin, which contains the logic for validating member status and processing loyalty points.

The Travel_Agent class acts as a controller that handles booking requests for HotelBooking, FlightBooking, and CruiseBooking.

TravelerInfo then links traveler profiles with specific reservations from the booking classes.

API Endpoints

Method	Purpose	URL	Notes
GET	Retrieve list of all available trips	/api/trips	Returns all trips from the database
GET	Retrieves information for a single trip	/api/trips/:tripCode	Returns single trip instance from the database, identified by the tripCode
POST	Creates and saves a new trip	/api/trips	Authenticated with JWT
PUT	Updates an existing trip	/api/trips/:tripCode	Updates a single trip instance in the database by the tripCode
POST	Creates a new user account	/api/register	Registers a new user
POST	Allows users to login to account	/api/login	Authenticates a user and returns a JWT
DELETE	Deletes an existing trip	/api/trips/:tripCode	Deletes a single trip instance from database, must be authorized user

The User Interface

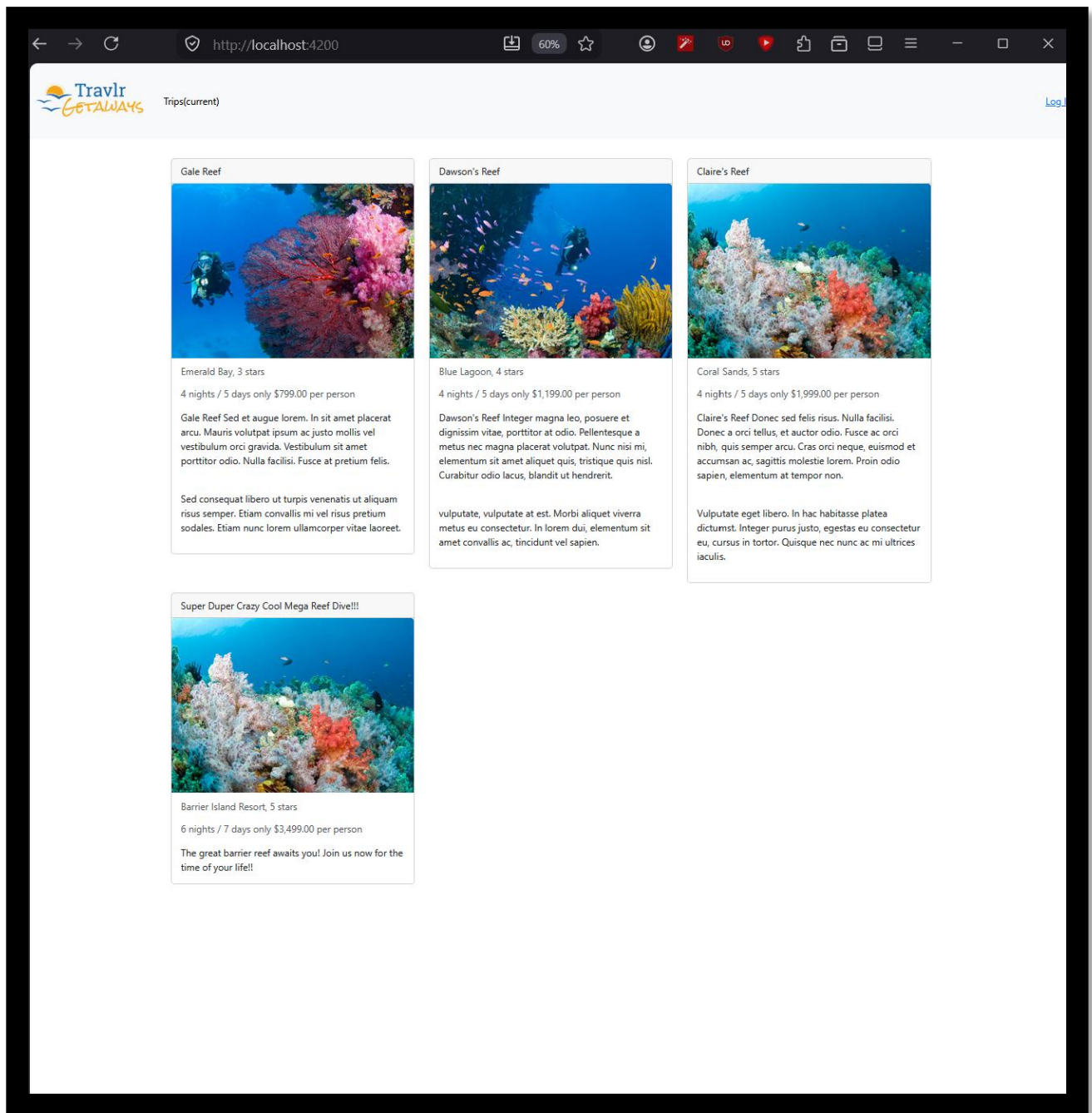


Figure 1: Admin Trips screen before logging in.

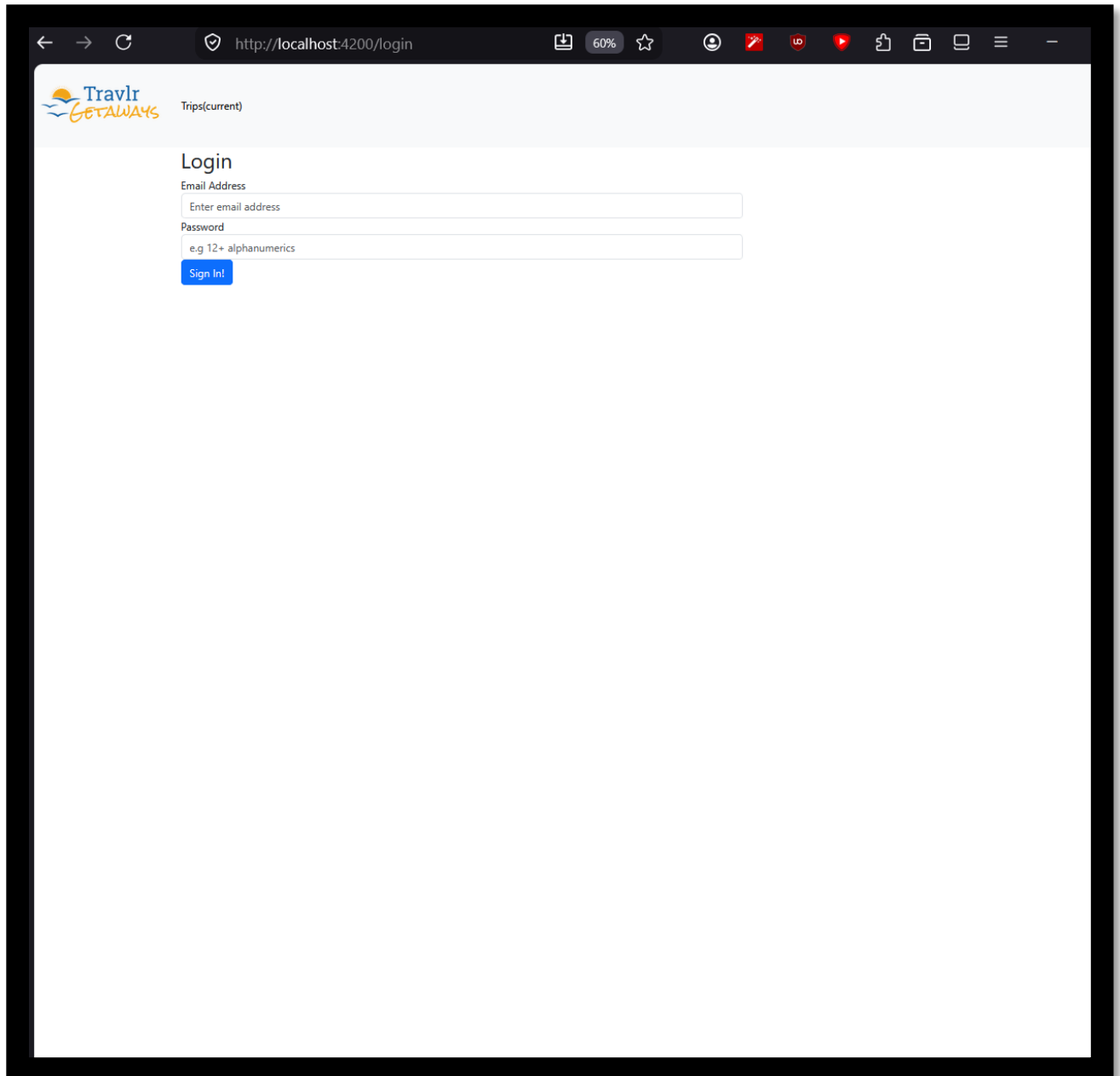


Figure 2: Admin login screen.

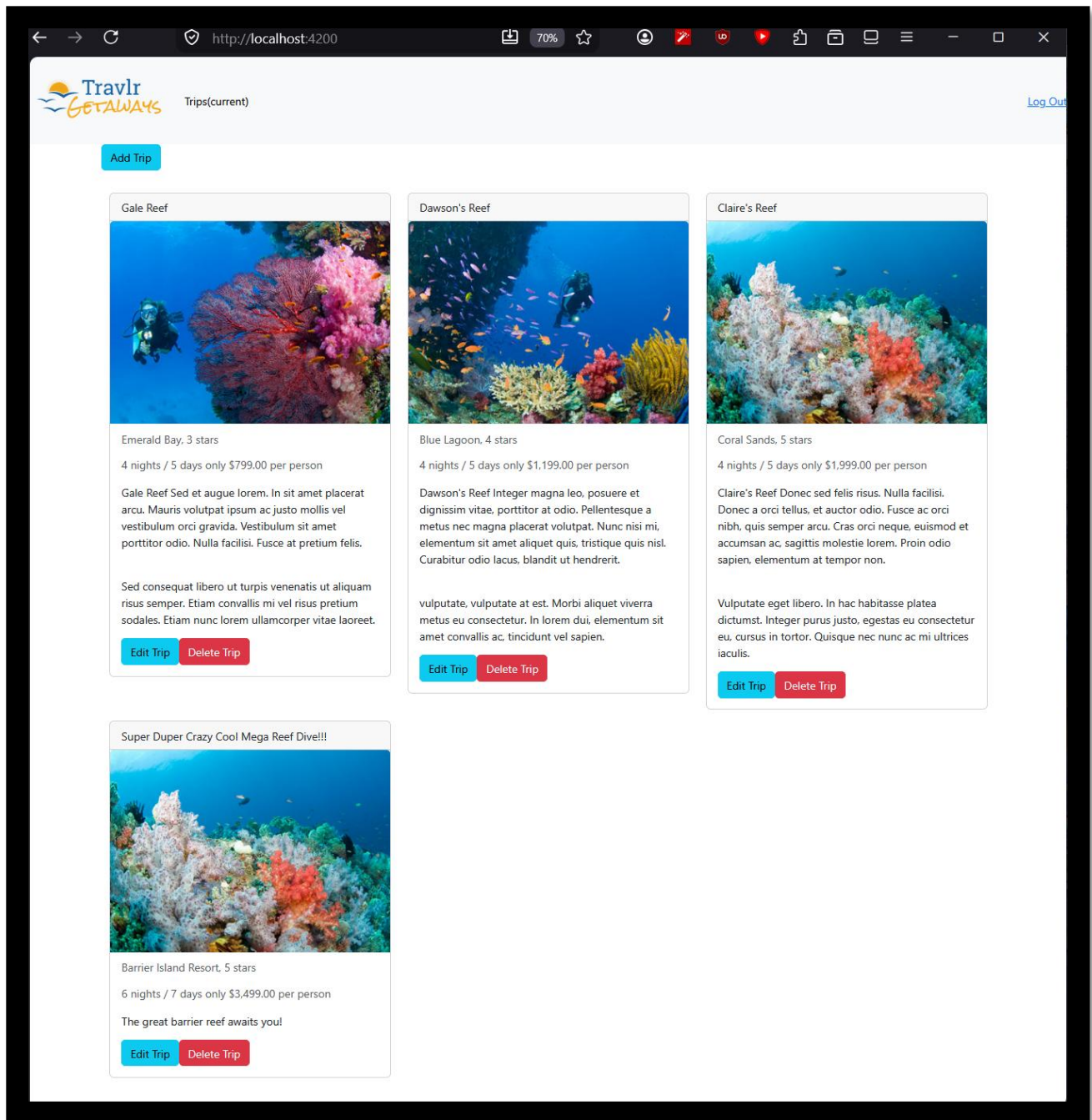


Figure 3: Admin trip screen after logging in successfully.

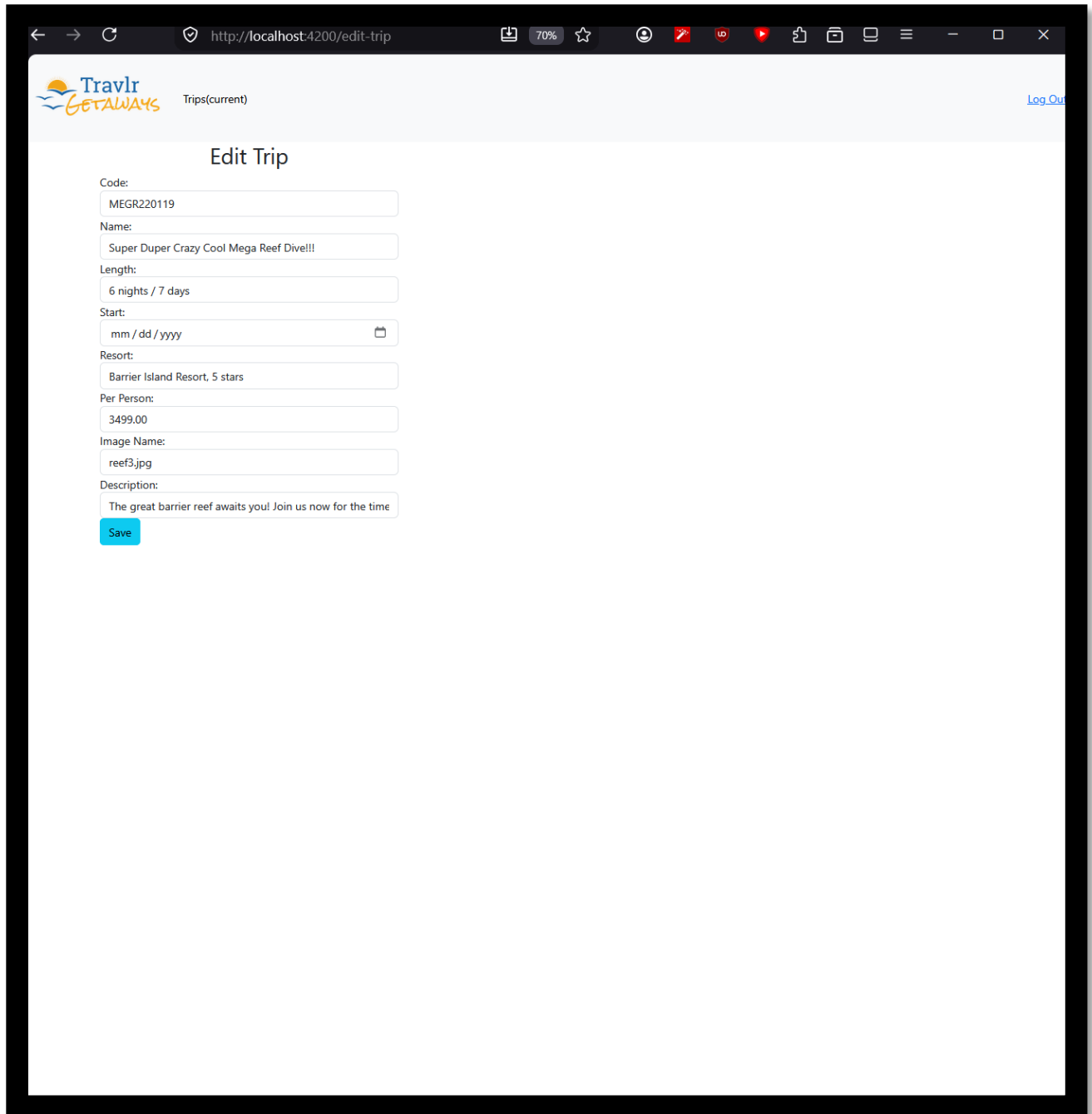


Figure 4: Edit trip screen displaying custom trip.

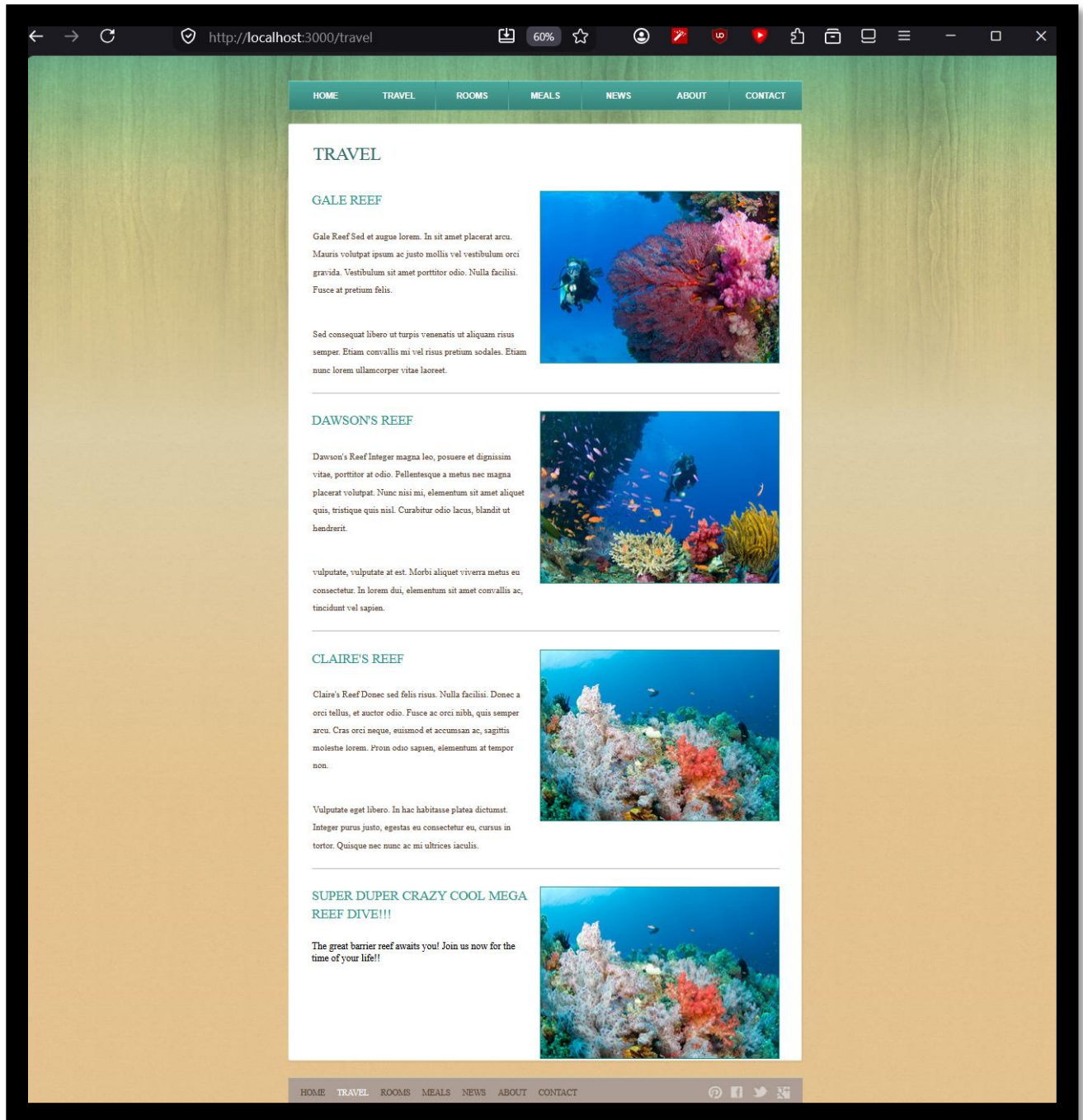


Figure 5: Customer-facing website displaying all available trips.

Single Page Application

Angular Project Structure VS Express HTML page

After working on Travlr Getaways web application I have gained experience and a better understanding of how the MEAN stack works. The customer pages use an Express server-side architecture and are organized using routes, controllers, views, and static files that are served through Node. When a customer requests a page, the server responds with a request and displays the complete HTML page in their browser. Every tab/page on the customer website requires another request and response from the server to serve the other HTML pages, which require full reloads.

The Angular admin page, on the other hand, uses single page application architecture. Angular initially loads the primary HTML, CSS, and JavaScript files but uses a client-side router for managing navigation while dynamically changing the display through components. This means that when an admin selects create trip or one of the edit buttons, the page does not have to perform a full reload.

Advantages and Disadvantages of SPA

The biggest advantage I noticed with the SPA was how much smoother the user experience felt. Since Angular can update components that are displayed without reloading, navigation is way smoother. Another notable advantage is that this approach creates a better separation between the front and backend of the application. Angular communicates with the API through Express to receive HTTP requests to receive JSON data instead of complete HTML pages like the customer-facing website.

A newcomer disadvantage for me was complexity. A traditional server rendered page can receive its data as part of the page loading process but with an SPA some components could render before the API has responded. This means it required careful consideration of loading states, empty collections, asynchronous responses, and possible errors. I experienced this complexity when I had to research a way to get the trip cards to continue display while I was actively building the page. I overcame this challenge by importing Angular's `ChangeDetectorRef` and implementing it in the `getStuff()` method, to force a manual change detection.

The SPA did provide additional functionality like reactive forms, which simple web apps do not. These allow the application to validate and update information without requiring a full-page submission. The client-side router also provided additional functionality by allowing navigation without reloading the whole page. The SPA also utilized Injectable services which provided a centralized location for HTTP communications, keeping the API out of its individual components.

Testing Process of SPA, API, and Database

Testing the Angular application required a multifaceted approach. I used my browsers built in inspector, Postman, and MongoDB Compass. I started by testing the backend API endpoints by using Postman and the corresponding GET, PUT, POST and DELETE commands to verify they were working as expected. When testing the POST, PUT, and DELETE commands I was able to verify if the changes were updating, if a new trip was being added, or records were being removed as expected through MongoDB Compass. Lastly, I tested editing and creating a new trip through the SPA, which verified that the whole app was working as expected. Testing and debugging through this whole build process helped me understand the

many places a failure could occur during the development of an SPA and how to verify each part is working as intended.

As with any project I am working on, I ran into problems. The biggest problem I ran into was the one I mentioned earlier. The trip card components would quit rendering while I was working on other parts of the application. The browser inspector was detecting all the trips and did not display any errors. So, I knew something had to go on with the rendering. I scoured the web and found that angular has a `ChangeDetectorRef` that I imported and implemented in the `getStuff()` method to force the changes to apply every time I saved a file.

I ran into some standard problems like simple syntax errors or accidental pluralities. Thankfully, VS Code is good at catching typos, and the browser console was good at telling me where problems were, so I could go investigate and fix them. Overall, these problems strengthened my debugging skills while giving me an understanding of how all the data moves through the entire web application.